

StockNet - Parallelized Neural Network for Stock Price Prediction Using High-Performance Computing Resources

Benjamin Zheng

Henry Odza

Prishita Ghanathe Krishna

Ronald Han

Toby Cowles

Abstract

In this project, we aim to design and implement a stock price predictor using the HPC cluster. Our approach will focus on the S&P 500 historical dataset, which provides a good source of financial data for model training and evaluation. We will begin by collecting and preprocessing this data to make sure it is clean, consistent, and suitable for modeling. Using the computational power of the Markov cluster, we will develop and train multiple neural network models to predict stock price movements and identify stocks likely to outperform market trends. These models will be designed to capture nonlinear relationships and temporal dependencies within the market.

1 Introduction

2 Datasets and Preprocessing

2.1 Datasets

Our analysis utilizes historical stock price data from the S&P 500 index, obtained from the Kaggle Stock Market Dataset. The dataset contains daily trading records for 500 stocks spanning multiple years, providing a comprehensive view of large-cap U.S. equities.

For each stock $s \in \{1, 2, \dots, S\}$ where S is the total number of stocks, we extract time series data consisting of:

$$\mathbf{x}_s^{(t)} = [L_t, O_t, V_t, H_t, C_t, A_t]^T \in \mathbb{R}^6$$

where for each trading day t :

- (1) L_t : Low price
- (2) O_t : Open price
- (3) V_t : Trading volume
- (4) H_t : High price
- (5) C_t : Close price
- (6) A_t : Adjusted close price

2.2 Preprocessing

To ensure temporal consistency across training samples, we impose a strict completeness requirement. For a stock s to be included in the training set, its data must satisfy:

$$\text{valid}(s) = \begin{cases} 1 & \text{if } |T_s| \geq T_{\min} \text{ and } \nexists t \in [1, T_s] : \mathbf{x}_s^{(t)} = \text{NaN} \\ 0 & \text{otherwise} \end{cases}$$

where T_s denotes the number of available time steps for stock s and $T_{\min} = 1000$ is our minimum sequence length requirement.

This filtering criterion serves two purposes:

- (1) **Temporal Consistency**: Ensures all stocks have equal sequence lengths for batch processing
- (2) **Data Integrity**: Eliminates stocks with missing values that would introduce bias

After applying this filter, our dataset reduces from 500 to S_{valid} stocks, where:

$$S_{\text{valid}} = \sum_{s=1}^S \text{valid}(s)$$

For each valid stock s , we extract the most recent $T = 1000$ consecutive trading days, creating a standardized input representation:

$$X_s = [\mathbf{x}_s^{(1)}, \mathbf{x}_s^{(2)}, \dots, \mathbf{x}_s^{(T)}] \in \mathbb{R}^{T \times 6}$$

This yields a corpus of stock sequences:

$$X = \{X_1, X_2, \dots, X_{S_{\text{valid}}}\}$$

where each X_s represents a complete 1000-day trading history for stock s .

Raw stock prices exhibit high variance across different equities due to fundamental value differences (e.g., penny stocks vs. high-value stocks). To stabilize training and ensure gradient flow, we apply feature-wise min-max normalization. For feature dimension $f \in \{1, 2, \dots, 6\}$, we first aggregate all values across stocks and time:

$$\mathbf{v}_f = \{X_{s,t,f} : s \in [1, S_{\text{valid}}], t \in [1, T]\}$$

The normalization transformation is then:

$$X_{s,t,f}^{\text{norm}} = \frac{X_{s,t,f} - \min(\mathbf{v}_f)}{\max(\mathbf{v}_f) - \min(\mathbf{v}_f)}$$

This maps all feature values to the range $[0, 1]$, ensuring:

$$X_{s,t,f}^{\text{norm}} \in [0, 1] \quad \forall s, t, f$$

All of this is done within a DataLoader object that is then passed onto the different models for use during training and evaluation.

3 Model Design and Implementation

3.1 Long Short Term Memory (LSTM) Network

One of the architectures we used to implement StockNet is the Long Short Term Memory (LSTM) network. LSTMs are a type of recurrent neural network (RNN) that are well-suited for sequence prediction problems due to their ability to capture long-term dependencies in data. In our implementation, we designed an LSTM network with multiple layers to effectively learn from the historical stock price data.

The complete architecture consists of:

- (1) A multi-layer LSTM encoder that processes historical price sequences
- (2) A fully connected output layer that maps LSTM hidden states to price predictions
- (3) An iterative forecasting mechanism that generates future price sequences based on previous predictions

Our implementation employs a stacked LSTM architecture with L layers, where $L = 2$ in the default configuration. For layer $\ell \in \{1, 2, \dots, L\}$, the hidden state at time t is denoted $h_t^{(\ell)}$. The output of layer $\ell - 1$ serves as input to layer ℓ :

$$h_t^{(\ell)} = \text{LSTM}^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}, c_{t-1}^{(\ell)})$$

where $h_t^{(0)} = x_t$ represents the input sequence. The hidden size is set to $h = 64$ neurons per layer.

To prevent overfitting, we apply dropout regularization between LSTM layers. During training, dropout randomly sets a fraction p of hidden state activations to zero:

$$h_t^{(\ell)} \leftarrow h_t^{(\ell)} \odot m^{(\ell)}$$

where $m^{(\ell)} \sim \text{Bernoulli}(1 - p)$ and $p = 0.2$ in our configuration.

At inference time, activations are scaled by $(1 - p)$ to maintain expected values. The LSTM's final hidden state $h_T^{(L)}$ at the last time step T is transformed to a scalar price prediction via a fully connected layer:

$$\hat{y}_{T+1} = W_{out} \cdot h_T^{(L)} + b_{out}$$

where $W_{out} \in \mathbb{R}^{1 \times h}$ and $b_{out} \in \mathbb{R}$ are learnable parameters. This yields the predicted normalized close price for the next time step. Raw stock price data undergoes feature-wise min-max normalization to stabilize training:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

where x_{min} and x_{max} are computed across all stocks and time steps in the training set. Normalized values lie in $[0, 1]$. We extract only the close price feature for autoregressive modeling, resulting in input sequences $X \in \mathbb{R}^{N \times (T-1) \times 1}$ where N is the number of stocks and $T = 1000$ is the sequence length.

For our loss function, we employ Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - \hat{y}_i)^2$$

where B is the batch size, y_i is the true normalized close price, $\hat{y}_i$ is the prediction, and θ represents all model parameters. The main idea being that MSE penalizes larger errors, encouraging the model to produce more accurate price predictions.

The Adam optimizer was used with learning rate $\alpha = 0.001$:

$$\theta_{t+1} = \theta_t - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon}$$

where m_t and v_t are exponentially weighted moving averages of gradients and squared gradients, respectively. The training process iterates for $E = 1000$ epochs with batch size $B = 32$. We shuffle training samples at each epoch to reduce overfitting to batch ordering.

As for evaluation, forecasting k steps ahead, we employ an iterative autoregressive approach. Starting with historical sequence $\mathbf{x} = [x_1, x_2, \dots, x_T]$, we predict:

- (1) $\hat{x}_{T+1} = f_{\theta}(\mathbf{x})$
- (2) $\hat{x}_{T+2} = f_{\theta}([x_2, x_3, \dots, x_T, \hat{x}_{T+1}])$
- (3) $\hat{x}_{T+k} = f_{\theta}([x_k, x_{k+1}, \dots, x_T, \hat{x}_{T+1}, \dots, \hat{x}_{T+k-1}])$

where f_{θ} represents the trained LSTM model. This sliding window approach maintains constant input sequence length while incorporating previously generated predictions.

Final predictions are transformed back to original price scale:

$$\hat{y}_{price} = \hat{y}_{norm} \times (x_{max} - x_{min}) + x_{min}$$

This allows direct interpretation of forecasted prices in currency units and facilitates comparison with actual market values.

3.2 Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) provide an alternative to recurrent models for sequence modeling by exploiting local temporal structure with convolutional filters. In our work we implement two variants: a 1D temporal CNN that operates along the time axis and returns one prediction per timestep (many-to-many), and a 2D CNN that treats time and feature dimensions as two spatial axes to explicitly model cross-feature interactions at each time window. Both variants consume the normalized input sequences $X \in \mathbb{R}^{T \times F}$ (with $T = 1000$ and $F = 6$ features per time step) and produce a predicted normalized close price for each time step in the output sequence.

3.2.1 1D Convolutional Model. The 1D model applies temporal convolutional filters across the sequence for each feature channel. Intuitively, a 1D convolution with kernel size k slides a filter over time and learns local temporal patterns such as short-term trends, local maxima/minima, or momentum that are predictive of the next close price. In our implementation the network expects inputs shaped as (batch, T , F) which are permuted to (batch, F , T) before feeding into PyTorch's Conv1d layers. The architecture is a small

stack of Conv1d blocks with ReLU activations and dropout for regularization, followed by a 1×1 projection to produce a single scalar per time step. Formally, if $\mathbf{x}_{b,:t} \in \mathbb{R}^F$ is the feature vector for batch index b at time t , the network computes:

$$\mathbf{h}_{b,:t}^{(1)} = \text{ReLU}(W^{(1)} * \mathbf{x}_{b,:t} + b^{(1)}),$$

where $*$ denotes a 1D convolution over the temporal axis and subsequent layers further mix temporal context. The final projection yields $\hat{y}_{b,t} \in \mathbb{R}$ for each t .

Key training and design choices for the 1D model:

- Many-to-many prediction: the model outputs a sequence $\hat{Y} \in \mathbb{R}^{T \times 1}$ so that intermediate predictions can be used for sequence-level losses and diagnostics. This stabilizes learning relative to single-step forecasting in some regimes.
- Loss and optimizer: Mean Squared Error (MSE) is used as the training objective and optimized with Adam ($\alpha = 10^{-3}$), matching the LSTM's configuration to allow fair comparison.
- Regularization: dropout is applied after each activation and the receptive field is controlled by kernel size and stacking depth. Padding is used to preserve sequence length so outputs align with inputs.
- Checkpointing: the trainer saves the model state when validation (or training) loss improves, enabling early stopping and safe resumption of training.

The 1D CNN is computationally efficient and benefits from parallelization on GPUs because convolutions are highly optimized. Its inductive bias favors temporal locality, making it especially effective at learning short-to-medium range temporal patterns with a small parameter count.

3.2.2 2D Convolutional Model. The 2D variant treats the sequence as a 2D image with axes time $\times$ features. Concretely, an input (batch, T , F) is reshaped to (batch, 1, T , F) and processed by Conv2d layers with kernels (k_t, k_f) that span both temporal and feature neighborhoods. After several Conv2d blocks the network projects to a single channel and reduces the feature axis (for example by averaging) so that the final output shape is (batch, T , 1).

This design explicitly models interactions between features at local temporal windows, enabling the network to learn patterns such as joint movements of open/high/low/close/volume that are predictive of future close prices. The forward pass can be summarized as:

$$\mathbf{Z} = \text{Conv2D}_{\text{stack}}(\text{reshape}(X)), \quad \hat{Y}_{b,t} = \text{pool}_f(\mathbf{Z}_{b,1,t,:}),$$

where pool_f denotes the operation that collapses the feature axis (e.g., mean over features) after the final 1-channel projection.

Advantages and trade-offs of the 2D model:

- Cross-feature coupling: 2D kernels can learn filters that respond to specific combinations of features (for instance, a high-volume up-tick pattern), which the 1D model can only approximate indirectly by combining per-feature channels.
- Larger parameter and compute cost: Conv2d layers typically require more memory and compute than Conv1d for the same effective receptive field, especially when using larger feature kernels k_f . This increases GPU memory usage and may slow training for long sequences like $T = 1000$.
- Flexible receptive field: by tuning (k_t, k_f) and stacking depth we can control the temporal and cross-feature scope of learned filters.

Multi-scale kernels or separable convolutions are straightforward extensions to reduce cost while preserving expressivity.

Both models are trained on the same normalized dataset and evaluated with identical metrics (MSE on held-out validation sequences) so that their statistical and computational trade-offs can be directly compared. In practice, the 1D model often trains faster and requires fewer parameters for comparable short-range forecasting accuracy, while the 2D model can outperform when cross-feature patterns are strong and worth the additional computational expense.

Practical improvements to consider (low-risk extensions): dilated convolutions or residual blocks to increase effective temporal context without proportional parameter growth; depthwise separable convolutions to reduce cost of 2D filters; and multi-scale branches that combine short and long kernels to capture patterns at multiple horizons. These ideas are easy to prototype within the existing trainer and model modules and can yield robust gains for stock time-series, which exhibit structure at many temporal scales.

4 Parallelization Strategy

5 Experiments

To evaluate the performance and scalability of our StockNet models, we conducted a comprehensive series of experiments across different computational configurations. Our experimental design focuses on understanding how model training scales with varying hardware resources, comparing both GPU-accelerated and CPU-based training approaches. The experiments were executed on the Markov HPC cluster, utilizing SLURM workload manager for resource allocation and job scheduling.

5.1 Experimental Setup

Our experimental framework encompasses three distinct model architectures: the LSTM network described in Section 3.1, a 1D Convolutional Neural Network (CNN1D), and a 2D Convolutional Neural Network (CNN2D). For each architecture, we systematically varied the computational resources to investigate scaling behavior and training efficiency. The experimental configurations span single GPU training, multi-GPU distributed training with 2 and 4 GPUs, and CPU-only training for baseline comparison.

All GPU experiments were conducted on the markov_gpu partition with NVIDIA GPUs, allocating 8 CPU cores and 32GB of memory per job. Multi-GPU experiments utilized PyTorch's Distributed Data Parallel (DDP) framework to enable efficient parallel training across multiple devices. Each training run was configured for a maximum of 1000 epochs with early stopping criteria, using a batch size of 64 for single GPU experiments and scaled batch sizes for multi-GPU configurations to maintain computational efficiency.

5.2 SLURM Job Configurations

To manage the diverse experimental conditions, we developed a suite of SLURM batch scripts that automate the training process across different resource allocations. Each script follows a standardized template that specifies job parameters, loads required modules, activates the Python environment, and executes the training script

with appropriate configuration flags. For example, our single GPU training script for the 1D CNN model includes:

```
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G

python main.py --mode train --model cnn
               --cnn_type 1d --epochs 1000
               --batch_size 64 --device cuda
```

Multi-GPU configurations extend this template by requesting additional GPU resources (`-gres=gpu:N`) and invoking PyTorch’s distributed launcher:

```
torchrun --nproc_per_node=N main.py
        --distributed ...
```

where $N \in \{2, 4\}$ specifies the number of GPUs. This distributed setup partitions the dataset across available GPUs, with each device processing a subset of the batch in parallel. Gradient synchronization occurs at each optimization step through all-reduce operations, ensuring model consistency across workers.

5.3 CPU Training Experiments

In addition to GPU-accelerated training, we conducted CPU-only experiments to establish performance baselines and understand the computational advantages provided by GPU hardware. CPU training jobs were submitted to standard compute partitions without GPU allocation, utilizing higher core counts (16-32 cores) to leverage multi-threading capabilities. These experiments are particularly valuable for:

- (1) Quantifying the speedup achieved through GPU acceleration
- (2) Assessing the feasibility of model training in resource-constrained environments
- (3) Identifying computational bottlenecks that benefit most from parallel processing

CPU experiments used identical hyperparameters and dataset configurations as their GPU counterparts, differing only in the device specification (`-device cpu`). Training times and memory utilization were recorded for direct comparison with GPU results.

5.4 Experimental Metrics

For each experimental configuration, we collected comprehensive performance metrics including:

- **Training Time:** Wall-clock time to complete the full training regimen
- **Epoch Duration:** Average time per training epoch across all iterations
- **GPU Utilization:** Device memory usage and compute utilization percentages
- **Scaling Efficiency:** Speedup ratio when moving from 1 to N GPUs, defined as $S(N) = \frac{T_1}{T_N}$ where T_N is training time with N GPUs
- **Model Accuracy:** Mean squared error on validation set across training checkpoints

All experiments were logged systematically with job-specific output files (`logs/model_Ngpu_%j.out`) containing training progress, loss curves, and resource utilization statistics. This logging infrastructure enables post-hoc analysis of convergence behavior and identification of optimal hardware configurations for each model architecture.

The complete experimental suite comprises 13 distinct job configurations spanning the three model types, four GPU configurations (1, 2, 4 GPUs, and CPU), and includes additional retraining experiments for model refinement. This comprehensive approach allows us to make evidence-based recommendations for resource allocation when deploying stock prediction models on HPC infrastructure.

6 Results